HW #4 (Parser Code Generation Complete Version)

COP 3402: Systems Software

Spring 2026

Instructor: Jie Lin, Ph.D.

Due date: **See Webcourses for the posted due date and time.**

Important: This handout is the primary written specification for the assignment and shall be read together with course lectures, demonstrations, Webcourses announcements, and any written clarifications issued by the instructor.

If any provision of this handout is unclear, incomplete, or reasonably subject to more than one interpretation, students should request clarification from the instructor before relying on a particular interpretation.

Any clarification or supplementary direction issued by the instructor in lecture, through Webcourses, or in written course communication shall govern the interpretation and administration of this assignment.

Pseudocode note: The pseudocode in Appendix G is a starting point and may not be 100% aligned with the required implementation. Your responsibility is to understand the grammar in Figure 1 first, then adapt details as needed before coding.

Contents

1 Assignment Overview **5**

2 Interpretation and Clarifications **5**

3 Implementation Procedure **6**
3.1 Integrated Scanner, Parser, and Code Generator (`parsercodegen_comp.c`) . 6

4 Grammar Specification **6**

5 Compilation Instructions **8**

6 Testing Instructions **8**
6.1 Requirements 8
6.2 Command-Line Usage Examples 9

7 Input and Output Specifications **9**
7.1 Input 9
7.2 Output 9
7.3 Error Handling Requirements 10
7.4 Required Error Messages 10
7.4.1 Scanning Error (Lexical Error) 10
7.4.2 Syntax Errors (Grammar Violations) 10
7.4.3 Important Notes on Error Messages 13

8 Submission Instructions **13**
8.1 Code Requirements 13
8.2 What to Submit 14
8.3 Submission Guidelines 15

9 Academic Integrity, AI Usage and Plagiarism Policy **16**
9.1 AI Usage Disclosure 16
9.2 Plagiarism Detection and Writing Standards 16

10 Submission Deadlines and Late Policy **16**

11 Grading **17**

Appendices **20**

A Instruction Set Architecture (ISA) **20**

B Recognition Rules and Token Definitions **23**
B.1 Recognition Rules . . . 23
B.1.1 Recognized Elements . . . 23
B.1.2 Ignored Elements . . . 24
B.2 Name Table . . . 25
B.3 TokenType Enumeration . . . 25
B.4 Lexical Error Reporting . . . 26

C Complete Example for Correct Input **28**
C.1 Input: PL/0 Source Code . . . 28
C.2 Terminal Output . . . 28
C.3 File Output: elf.txt . . . 30
C.4 How to Test Your Output . . . 30

D Scanning Error Example **32**
D.1 Input: PL/0 Source Code with Error . . . 32
D.2 Terminal Output . . . 32
D.3 File Output: elf.txt . . . 32
D.4 Key Principle . . . 32

E Syntax Error Example **33**
E.1 Key Distinction: Scanning vs. Syntax Errors . . . 33
E.2 Input: PL/0 Source Code with Syntax Error . . . 33
E.3 Terminal Output . . . 33
E.4 File Output: elf.txt . . . 34
E.5 Key Principle . . . 34

F Symbol Table Structure **35**
F.1 Recommended Symbol Table Structure . . . 35
F.2 Symbol Table Size . . . 36
F.3 Lexicographical Level Tracking (HW4) . . . 36
F.4 Marking Algorithm (HW4) . . . 36
F.5 Required Storage for Different Symbol Types . . . 37
F.5.1 For Constants (kind = 1) . . . 37
F.5.2 For Variables (kind = 2) . . . 37
F.5.3 For Procedures (kind = 3) . . . 37
F.6 Symbol Table Operations . . . 38

G Pseudocode **39**
G.1 Symbol Table Helper Function . . . 39
G.2 PROGRAM Function . . . 39
G.3 BLOCK Function . . . 39
G.4 CONST-DECLARATION Function . . . 40
G.5 VAR-DECLARATION Function . . . 40
G.6 PROCEDURE-DECLARATION Function . . . 41

G.7 STATEMENT Function 42
G.8 CONDITION Function 45
G.9 EXPRESSION Function 45
G.10 TERM Function 46
G.11 FACTOR Function 46

1 Assignment Overview

HW4 is a direct extension of Homework 3. In HW3 you built a recursive-descent parser and PM/0 code generator for a flat PL/0 grammar in which every symbol lived at lexicographical level 0 and there were no procedures. In HW4 you will extend that implementation to support:

1. **Procedure declarations** as part of every block, using the new `<procedure-declaration>` production (see Figure 1).
2. **Procedure calls** in statements via the new `"call" <ident>` statement form.
3. **Lexicographical level management** throughout the symbol table, parser, and code generator. Symbols declared inside a nested procedure must record the correct level, and references to those symbols must compute a non-zero L field for `LOD`, `STO`, and `CAL` instructions.
4. **Symbol marking (scope exit).** When a procedure's block finishes parsing, the symbols declared within that block must be marked so that they can no longer be referenced from outside their enclosing scope.

You will submit a **single, integrated program** called `parsercodegen_comp.c` that incorporates your HW2 scanner logic (from `lex.c`) together with the HW3 parser/code generator extended with the HW4 features listed above. This program reads a PL/0 source file directly, tokenizes it internally, parses the token stream according to the grammar in Figure 1, and—only when no errors occur—generates executable PM/0 assembly code in the ISA format specified in Appendix A.

What is new compared to HW3. The HW3 deliverable emitted PM/0 code in which every `LOD` and `STO` instruction used $L = 0$, and the `CAL` and `RTN` instructions were never generated. In HW4 these restrictions are removed: your generator must correctly produce L values that reflect the difference in nesting level between the reference site and the declaration site of each symbol, and it must emit `CAL` and `RTN` for procedure calls and procedure returns respectively.

2 Interpretation and Clarifications

This handout constitutes the primary written assignment specification. It shall be interpreted together with course lectures, live demonstrations, Webcourses announcements, and any written clarifications issued by the instructor for this assignment.

No written assignment document can anticipate every possible implementation question or factual scenario. Accordingly, if any provision is unclear, incomplete, or reasonably susceptible to more than one interpretation, students should seek clarification from the instructor before submission.

For purposes of interpretation, grading, and administration of the assignment, any clarification, limitation, example, or supplementary direction subsequently issued by the instructor in lecture, through Webcourses, or in written course communication shall govern.

3 Implementation Procedure

This assignment requires a **single** C source file.

3.1 Integrated Scanner, Parser, and Code Generator (`parsercodegen_comp.c`)

- Create `parsercodegen_comp.c` by extending your HW3 `parsercodegen.c`. You should start from a working HW3 solution (either your own HW3 submission or an instructor-provided reference) and add the HW4 features described in Section 1.
- All scanner, parser, symbol-table, and code-generator logic must live in the single file `parsercodegen_comp.c`. Do not split the implementation across multiple `.c` files.
- `parsercodegen_comp.c` must accept exactly **ONE** command-line argument: the path to the PL/0 source file.
- The scanner runs internally and produces a token list in memory.
- After scanning, the parser reads the in-memory token list, validates the grammar in Figure 1, performs lexicographical level tracking across procedure declarations, and generates PM/0 assembly code (see Appendix A for the ISA specification).
- The generated code is written to `elf.txt` in numeric format.

4 Grammar Specification

Your parser must accept exactly the productions in Figure 1. In EBNF notation, nonterminals are enclosed in angle brackets (e.g., `<program>`, `<statement>`), while terminals are keywords, symbols, or literals shown in quotes (e.g., `"const"`, `":="`).

EBNF Notation Review

- `[ ... ]` means the enclosed part is optional and may appear zero or one time.
- `{ ... }` means the enclosed part may repeat zero or more times.
- `( ... )` groups symbols together so they are treated as one unit.
- `|` means "or" and separates alternatives.

PL/0 Grammar (EBNF)

```
<program> ::= <block> "."

<block> ::= <const-declaration> <var-declaration>
            <procedure-declaration> <statement>

<const-declaration> ::= [ "const" <ident> "=" <number>
                          { "," <ident> "=" <number> } ";" ]

<var-declaration> ::= [ "var" <ident> { "," <ident> } ";" ]

<procedure-declaration> ::= { "procedure" <ident> ";" <block> ";" }

<statement> ::= [ <ident> ":=" <expression>
                | "call" <ident>
                | "begin" <statement> { ";" <statement> } "end"
                | "if" <condition> "then" <statement>
                  [ "else" <statement> ] "fi"
                | "while" <condition> "do" <statement> "od"
                | "read" <ident>
                | "write" <expression> ]

<condition> ::= <expression> <rel-op> <expression>

<expression> ::= ["-"] <term> { ("+" | "-") <term> }

<term> ::= <factor> { ("*" | "/" ) <factor> }

<factor> ::= <ident> | <number> | "(" <expression> ")"

<number> ::= <digit> { <digit> }
<ident> ::= <letter> { <letter> | <digit> }
<rel-op> ::= "=" | "<>" | "<" | "<=" | ">" | ">="

<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<letter> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"
```

Figure 1: Grammar to be implemented by the HW4 parser. New productions compared with HW3 are highlighted by the addition of `<procedure-declaration>` inside `<block>` and the `"call" <ident>` alternative inside `<statement>`.

Grammar changes from HW3 to HW4. For quick reference, the grammar differs from HW3 in exactly the following places:

- `<block>` now contains `<procedure-declaration>` between `<var-declaration>` and `<statement>`. An empty procedure-declaration is legal (zero repetitions of the outer braces), so HW3-style programs without procedures are still accepted.
- `<procedure-declaration>` is a new production whose body is itself a `<block>`. This allows arbitrarily nested procedure declarations, which is the reason lexicographical level management is required.
- `<statement>` has a new alternative, `"call" <ident>`, which invokes a previously declared procedure and must generate a `CAL` instruction.

Reserved words, symbols, and token type names must exactly match the definitions provided in HW2. Identifiers and numbers retain the lexemes emitted by the scanner.

5 Compilation Instructions

Compile and run on **Eustis**. The only permitted language is C.

Compilation Commands

```
// Compile the integrated scanner/parser/code generator
gcc -O2 -Wall -std=c11 -o parsercodegen_comp parsercodegen_comp.c
```

Note: The only permitted language is C. All logic for HW4 (scanner, parser, symbol table with lexicographical level management, and code generator) resides in the single `parsercodegen_comp.c` file.

Do not require additional libraries beyond the C standard library. Any makefiles or helper scripts must preserve the same compiler flags and target names.

6 Testing Instructions

Invoke programs from the terminal/command line on **Eustis** (the university grading server).

6.1 Requirements

- `parsercodegen_comp.c` must accept exactly **ONE** command-line parameter: the path to the PL/0 source program.
- Do not prompt for input.
- Reject incorrect argument counts with a helpful usage message and exit.
- The scanner runs internally and tokenizes the source in memory.

- The parser reads the in-memory token list, validates the grammar, performs lexicographical level tracking across nested procedure declarations, and generates PM/0 assembly code.
- Never request additional input from the user.

6.2 Command-Line Usage Examples

Usage Examples

```
./parsercodegen_comp inputfile.txt
```

Explanation: The single `parsercodegen_comp` executable reads the PL/0 source file, scans it, parses the token stream, generates code with proper lexicographical level tracking, and produces all output.

7 Input and Output Specifications

This section describes the exact input and output requirements. For complete working examples, see Appendix C (correct input), Appendix D (scanning error), and Appendix E (syntax error).

7.1 Input

- **Input file:** A PL/0 source program provided as a command-line argument
- **Format:** Plain text file containing PL/0 source code
- **Example:** `./parsercodegen_comp input.txt`
- **See:** Appendix C for a complete example of correct PL/0 input

7.2 Output

- **Terminal output:**
  - If no errors: Display generated assembly code with line numbers and symbol table
    - **See:** Appendix C for an example of correct terminal output
  - If errors detected: Display appropriate error message (see Section 7.4)
    - **See:** Appendix D for scanning error output
    - **See:** Appendix E for syntax error output
- **File output (`elf.txt`):**

- If no errors: Write assembly code in numeric format (one instruction per line: `OP L M`)
  - **See:** Appendix C for an example of correct `elf.txt` output
- If errors detected: Write the same error message as displayed on terminal
  - **See:** Appendix D and Appendix E for error output examples

7.3 Error Handling Requirements

Your program must detect and report errors appropriately. When an error is detected, the program must:

1. Stop processing immediately (do not attempt to continue parsing or code generation)
2. Output the appropriate error message to the terminal
3. Write the same error message to `elf.txt`
4. Exit gracefully

7.4 Required Error Messages

Your `parsercodegen_comp.c` implementation must support the required scanning and syntax-error situations described below.

7.4.1 Scanning Error (Lexical Error)

Scanning Error Message - Minimum Requirement

Suggested message:
`Error: Scanning error detected by lexer (skipsym present)`

When to output: Token type 1 (skipsym) is found in the token list
Meaning: The scanner detected a lexical error (identifier too long, number too long, or invalid symbol)
Complete Example: See Appendix D for a detailed walkthrough.

7.4.2 Syntax Errors (Grammar Violations)

The following error messages correspond to violations of the PL/0 grammar rules. See Appendix E for a complete example of syntax error handling. For readability, the required-error list is split into two non-breakable boxes: one for the 17 HW3 errors that carry over unchanged, and one for the 3 new HW4 errors.

Syntax Error Messages - General Notes

Very Important: There is **no autograder** for this class.

Very Important: The two lists below cover **all of the required error-message cases** that your implementation must support at a minimum.

Your `parsercodegen_comp.c` implementation must support at least the following syntax-error situations. The wording does **not** need to match exactly as written below. Exact capitalization, exact phrasing, and exact word-for-word output are **not required**, as long as your message clearly communicates the corresponding error.

HW3 Syntax Errors (17, carried over unchanged)

The following 17 errors are the same as in HW3 and must still be detected in HW4.

1. `Error: program must end with period`
   - **Complete Example:** See Appendix E for detailed walkthrough
2. `Error: const, var, and read keywords must be followed by identifier`
3. `Error: symbol name has already been declared`
4. `Error: constants must be assigned with =`
5. `Error: constants must be assigned an integer value`
6. `Error: constant and variable declarations must be followed by a semicolon`
7. `Error: undeclared identifier`
8. `Error: only variable values may be altered`
9. `Error: assignment statements must use :=`
10. `Error: begin must be followed by end`
11. `Error: if must be followed by then`
12. `Error: while must be followed by do`
13. `Error: do must be followed by od`
14. `Error: if-then statement must end with fi`
15. `Error: condition must contain comparison operator`
16. `Error: right parenthesis must follow left parenthesis`
17. `Error: arithmetic equations must contain operands, parentheses, numbers, or symbols`

HW4 Syntax Errors (3, new in HW4)

The following errors are introduced by the new `<procedure-declaration>` production and the `"call" <ident>` statement alternative. They must be detected in addition to the 17 HW3 errors above.

18. `Error: procedure and call keywords must be followed by identifier`
19. `Error: procedure declarations must be followed by a semicolon`
20. `Error: call must be followed by a procedure identifier`

Important: The 17 HW3 errors and the 3 HW4 errors above are the **minimum required syntax errors** that you are expected to catch and that we may test for. You may need to create **additional error messages** beyond this list in order to catch all malformed input programs correctly. In particular, HW4 introduces new failure modes around procedure declarations (nested declarations, scope violations, calls to undeclared procedures) that may require additional custom error messages.

7.4.3 Important Notes on Error Messages

- Using `Error:` at the beginning of an error message is recommended, but exact wording is not required.
- Error messages do **not** need to be case-sensitive or match the sample text exactly.
- At minimum, your implementation must detect all of the required error situations listed in this section.
- When an error is detected, output the error message to both terminal and `elf.txt`
- Do not output multiple error messages—stop at the first error encountered
- You may need additional custom error messages beyond the list above to handle all invalid programs robustly.

8 Submission Instructions

Submit on **Webcourses**. Programs are compiled and tested on **Eustis**. Follow these to avoid deductions.

8.1 Code Requirements

- **Program name.** Submit exactly one source file: `parsercodegen_comp.c`.

- **Header comment (copy/paste).** Place this header comment at the top of your source file.

Required Header Comment

```
/*
Assignment:
HW4 - Parser/Code Generator Complete Version for PL/0
      (procedures + lexicographical level management)

Author(s): <Full Name 1>, <Full Name 2>

Language: C (only)

To Compile:
  gcc -O2 -Wall -std=c11 -o parsercodegen_comp parsercodegen_comp.c

To Execute (on Eustis):
  ./parsercodegen_comp <input_file.txt>

where:
  <input_file.txt> is the path to the PL/0 source program

Notes:
  - Single integrated program: scanner + parser + code gen
  - parsercodegen_comp.c accepts ONE command-line argument
  - Scanner runs internally (no intermediate token file)
  - Implements recursive-descent parser for the full PL/0 grammar
    including procedure declarations and the call statement
  - Tracks lexicographical levels (nesting depth) for every symbol
  - Generates PM/0 assembly code with proper L fields for LOD/STO/CAL
    and emits CAL/RTN instructions for procedures
    (see Appendix A for the ISA)
  - All development and testing performed on Eustis

Class: COP 3402 - Systems Software - Spring 2026

Instructor: Dr. Jie Lin

Due Date: See Webcourses for the posted due date and time.
*/
```

8.2 What to Submit

- Your source code file (`parsercodegen_comp.c`)
- A required set of **3 error-case pairs** demonstrating that your program correctly

catches the **new** HW4 errors introduced by the procedure-declaration and call grammar extensions. *Do not submit error-case pairs for the errors that were already required in HW3* — those will still be manually spot-checked by the grader, but submitting all of them again would bloat the submission system.

- Each error-case pair consists of:
  - one PL/0 input file that triggers a specific error case, and
  - one corresponding expected-output file showing the error message/output for that case.
- The 3 required error-case pairs must cover, at a minimum, the following HW4-specific errors from Section 7.4:
  1. `procedure and call keywords must be followed by identifier` — triggered by a `procedure` or `call` keyword that is not immediately followed by an identifier token.
  2. `procedure declarations must be followed by a semicolon` — triggered by omitting the trailing `;` after a procedure's body.
  3. `call must be followed by a procedure identifier` — triggered by calling an identifier whose symbol-table entry is not a procedure (e.g., a variable or constant).
- In total, this submission component contains **6 files**: 3 PL/0 input files plus 3 corresponding expected-output files.

- **If you used AI:** Submit the AI Usage Disclosure Form *and* a separate markdown file describing your AI usage.
- **If you did not use AI:** Submit no additional files.

8.3 Submission Guidelines

- Submit on Webcourses before the due date. Late submissions incur penalties as described below. Resubmissions are not accepted after two days.
- Do not modify the grammar or deviate from the grammar specification required for this assignment. Your `parsercodegen_comp.c` implementation must adhere to the HW4 grammar shown in Figure 1, including the new `<procedure-declaration>` production and the `"call" <ident>` statement alternative. If your implementation changes the grammar, omits required grammar rules, adds unsupported grammar behavior, or otherwise does not follow the specified grammar, it will be treated as plagiarism and will receive a zero.
- Only C is allowed for this assignment.

9 Academic Integrity, AI Usage and Plagiarism Policy

9.1 AI Usage Disclosure

If you use AI tools while completing this assignment, you must disclose this usage. Complete the **AI Usage Disclosure Form** provided with this assignment. If you used AI, include a separate markdown file describing:

- The name and version of the AI tool used.
- The dates used and specific parts of the assignment where the AI assisted.
- The prompts you provided and a summary of the AI output.
- How you verified the AI output against other sources and your own understanding.
- Reflections on what you learned from using the AI.

If you used AI and did not submit both the disclosure form and the markdown file, it will be treated as plagiarism and will result in a 0 for this submission.

9.2 Plagiarism Detection and Writing Standards

All submissions will be processed through plagiarism detection tools (e.g., JPlag). If the similarity score between your submission and others exceeds a threshold, your code will be considered plagiarized and you will receive an F for the course. This applies regardless of whether you used AI and regardless of whether you submitted an AI disclosure form. **If AI usage is detected and the required AI disclosure form/markdown are missing, the submission will be treated as plagiarism and receive a zero.**

While AI tools may assist with brainstorming or high-level guidance, the final submission must represent your own work and understanding. **Do not copy and paste AI-generated code.** AI tends to produce highly similar solutions across students; use it only to understand ideas, then implement the code yourself. Do not copy previous semester solutions or share your code with others.

10 Submission Deadlines and Late Policy

All deadlines use U.S. Eastern Time (Orlando, FL). The Webcourses submission timestamp is authoritative. A submission is considered late if it is uploaded after the posted due date/time on Webcourses. Late submissions are accepted for up to 48 hours after the Webcourses due date, subject to the following point penalties (deductions are applied to the assignment's maximum score, not a percentage):

- 0:00:01-12:00:00 late $\rightarrow -5$ points
- 12:00:01-24:00:00 late $\rightarrow -10$ points

- 24:00:01-36:00:00 late → −15 points
- 36:00:01-48:00:00 late → −20 points
- After 48:00:00 → Not accepted; recorded as missed (0 points)

Resubmissions after 48 hours are not accepted.

11 Grading

Your assignment will be graded based on the correctness of your parser/code generator and adherence to the assignment specification. This homework will be graded through manual review of your submitted code, compilation, execution behavior, and generated `elf.txt` output. Deductions are listed in one integrated list, from highest to lowest.

- **-100 points (Immediate zero):** Any one of the following actions:
  1. Plagiarism, resubmission of old programs, or direct copy-paste of AI-generated code or another student's work. Submissions are processed through JPlag; high-similarity results may trigger manual verification.
  2. Missing or incorrectly updated required header comments, including the required author names of all group members and related assignment metadata.
  3. Missing or incomplete AI Usage Disclosure Form.
  4. Missing required C source file `parsercodegen_comp.c`.
  5. Submitting the program under a filename other than `parsercodegen_comp.c`.
  6. Program is written in a language other than C.
  7. Program does not compile on Eustis using the required command and flags.
  8. Program does not accept exactly one command-line argument as required.
  9. Program cannot read the required input file.
  10. Program does not produce the required `elf.txt` output file.
  11. Program is not a working implementation of the required grammar.
  12. **(HW4)** Implementation that does not follow the specified PL/0 grammar in Figure 1 (e.g., omits `<procedure-declaration>`, omits the `"call" <ident>` statement alternative, or alters any production).
- **-15 points:** Major code generation errors in core instructions, including incorrect `LOD` or `STO` handling, or comparable errors that make the generated PM/0 code substantially incorrect.
- **-10 points:** `elf.txt` output is significantly incorrect or deviates from the required ISA/output format in Appendix A.

- **-10 points (HW4):** Errors in `LOD` (load) and `STO` (store) instruction generation — for example, incorrect L field (wrong lexicographical level difference) or incorrect M field (wrong address) for variable accesses across nested procedures.
- **-10 points (HW4):** Incorrect marking algorithm implementation. Every *variable* and every *nested procedure* declared inside a block must be marked (`mark = 1`) when that block finishes, so that they can no longer be referenced from enclosing scopes. Constants are *not* marked.
- **-5 points per issue:** Grammar implementation errors that cause parsing or code generation behavior to deviate from the assignment specification.
- **-5 points per issue:** Incorrect PM/0 instruction generation compared to expected results.
- **-5 points per issue:** Symbol table implementation errors, including incorrect value, level, address, or marking fields.
- **-5 points (HW4):** Incorrect `val` field for variable symbols or procedure symbols. The `val` field is *not used* for either variables or procedures; variables use `addr` for their activation-record offset and procedures use `addr` for the starting code address (see Appendix F).
- **-5 points (HW4):** Incorrect lexicographical level for symbols. Your implementation must properly track nesting depth with procedures: symbols declared in the main block are at level 0, symbols declared in a first-level nested procedure are at level 1, and so on.
- **-5 points (HW4):** Incorrect address calculation for variables. Each variable's `addr` field must account for the fixed activation-record header (see the pseudocode for `BLOCK/VAR-DECLARATION` in Appendix G).
- **-5 points per issue (HW4):** Any other symbol table implementation errors not covered by the items above.
- **-5 points:** Incorrect or missing error messages for invalid input.
- **-5 points:** Required scanner errors are not correctly detected or not correctly propagated into parser/code generator behavior.
- **-5 points:** Missing or incomplete required 3 HW4-specific test case file pairs (see Section 8.2).
- **-5 points per issue:** Any implementation issue identified during manual grading and code review that requires grader intervention, correction, or interpretation in order to evaluate your program. Each error fixed by the graders during grading is subject to a 5-point deduction.

Final grade calculation: Start with 100 points and subtract all applicable deductions listed above.

Note on Late Submissions: Late penalties are applied separately according to Section 10.

A Instruction Set Architecture (ISA)

The PM/0 virtual machine supports nine opcodes. Each instruction is encoded by a three-number tuple $\langle OP, L, M \rangle$. The tables below summarize each opcode along with a brief description and pseudocode. See Table 2 for `OPR` sub-operations.

Note: Your parser and code generator must emit instructions in this ISA format. The generated assembly code file should contain one instruction per line in the format: `OP L M`.

Pseudocode note: The pseudocode in this appendix is a starting point and may not be 100% aligned with the required implementation. Your responsibility is to understand the grammar in Figure 1 first, then adapt details as needed before coding.

Table 1: PM/0 Instruction Set (Core)

Opcode	Mnemonic	L	M	Description	Pseudocode
01	LIT	0	n	**Push literal.**	`sp` $\leftarrow$ `sp` $+1$ `pas[sp]` $\leftarrow$ n
02	OPR	0	m	**Operation code.**	*See OPR Table 2 for operation details*
03	LOD	n	a	**Load to stack.**	`sp` $\leftarrow$ `sp` $+1$ `pas[sp]` $\leftarrow$ `pas[base(bp,`n`)` $+a$`]`
04	STO	n	o	**Store from stack.**	`pas[base(bp,`n`)` $+o$`]` $\leftarrow$ `pas[sp]` `sp` $\leftarrow$ `sp` -1

05	CAL	n	a	**Call procedure.**	`pas[sp+1]` $\leftarrow$ `base(bp,`n`)`
					`pas[sp+2]` $\leftarrow$ `bp`
					`pas[sp+3]` $\leftarrow$ `pc`
					`bp` $\leftarrow$ `sp+1`
					`pc` $\leftarrow$ a

06	INC	0	n	**Allocate locals.**	`sp` $\leftarrow$ `sp` $+n$
07	JMP	0	a	**Unconditional jump**	`pc` $\leftarrow$ a
08	JPC	0	a	**Conditional jump.**	`if pas[sp]` $=$ `0 then pc` $\leftarrow$ a `sp` $\leftarrow$ `sp` -1
09	SYS	0	1	**Output integer.**	`print(pas[sp])` `sp` $\leftarrow$ `sp` -1
09	SYS	0	2	**Read integer.**	`sp` $\leftarrow$ `sp` $+1$ `pas[sp]` $\leftarrow$ `readInt()`
09	SYS	0	3	**Halt the program.**	`halt`

Table 2: PM/0 Arithmetic and Relational Operations (OPR, opcode 02, L=0)

Opcode	Mnemonic	L	M	Description	Pseudocode
02	RTN	0	0	**Return from subroutine.**	sp ← bp −1 bp ← pas[sp+2] pc ← pas[sp+3]
02	NEG	0	1	**Negation (unary minus).**	pas[sp] ← −pas[sp]
02	ADD	0	2	**Addition.**	pas[sp−1] ← pas[sp−1] + pas[sp] sp ← sp −1
02	SUB	0	3	**Subtraction.**	pas[sp−1] ← pas[sp−1] − pas[sp] sp ← sp −1
02	MUL	0	4	**Multiplication.**	pas[sp−1] ← pas[sp−1] ∗ pas[sp] sp ← sp −1
02	DIV	0	5	**Integer division.**	pas[sp−1] ← pas[sp−1] / pas[sp] sp ← sp −1
02	EQL	0	6	**Equality comparison**	pas[sp−1] ← (pas[sp−1] == pas[sp]) sp ← sp −1
02	NEQ	0	7	**Inequality comparison**	pas[sp−1] ← (pas[sp−1] ≠ pas[sp]) sp ← sp −1
02	LSS	0	8	**Less-than comparison**	pas[sp−1] ← (pas[sp−1] < pas[sp]) sp ← sp −1
02	LEQ	0	9	**Less-or-equal comparison**	pas[sp−1] ← (pas[sp−1] ≤ pas[sp]) sp ← sp −1
02	GTR	0	10	**Greater-than comparison**	pas[sp−1] ← (pas[sp−1] > pas[sp]) sp ← sp −1
02	GEQ	0	11	**Greater-or-equal comparison**	pas[sp−1] ← (pas[sp−1] ≥ pas[sp]) sp ← sp −1

Important (HW4 changes from HW3): Unlike HW3, HW4 programs may contain nested procedure declarations. Your code generator must therefore use the full ISA:

- `CAL` **is required.** Every `"call" <ident>` statement must emit a `CAL` instruction whose L field is the lexicographical level difference between the calling site and the declaration site of the target procedure, and whose M field is the code address of that procedure's first instruction.
- `RTN` **is required.** Every procedure body must end with a `RTN` instruction (encoded as `OPR 0 0`) so that control returns to the caller.
- `LOD` **and** `STO` **may use non-zero** L**.** When a statement references a variable declared in an enclosing (but not the current) procedure, the L field of the generated `LOD`

or `STO` must be the difference in lexicographical level between the reference and the declaration. If the reference is in the same block as the declaration, $L = 0$ as in HW3.

- The M field of `LOD` and `STO` remains the variable's `addr` from the symbol table.

B Recognition Rules and Token Definitions

This appendix collects the HW2 lexical definitions that remain in effect for HW4. Use these recognition rules, token numbers, and lexical-error requirements exactly as specified here. The reserved words `procedure` and `call`, which were unused in HW3 code generation, are now semantically active and drive the new grammar productions introduced in HW4.

B.1 Recognition Rules

The lexical analyzer recognizes the following elements and ignores others.

B.1.1 Recognized Elements

Skip Symbol (`skipsym`) Token number 1 is reserved as the skip symbol, `skipsym`. It is an internal placeholder/error token, not a reserved word in the PL/0 language. When the lexical analyzer encounters an invalid character or malformed token, it should output token `1` (`skipsym`).

For this reason, the text `skip` is not a language keyword and should not be recognized as a token by itself.

Identifiers and Numbers An identifier begins with a letter and may be followed by letters or digits. The maximum length is eleven characters. A number consists of one or more digits with a maximum length of five digits. Token number 1 is the skip symbol (`skipsym`), token number 2 is the identifier symbol (`identsym`), and token number 3 is the number symbol (`numbersym`). See Table 3 for details.

Table 3: Identifier and Number Token Definitions

Token Type	Token Symbol	Token Number	Regex Representation	Constraints
Identifier	identsym	2	`[a-zA-Z][a-zA-Z0-9]*`	Max 11 characters
Number	numbersym	3	`[0-9]+`	Max 5 digits

Reserved words See Table 4 for the complete list of reserved words. Token numbers 4 and onward are reserved words.

Table 4: Reserved Words Table

Symbol	Token Symbol	Token Number	Meaning/String Match
begin	beginsym	4	Reserved word
end	endsym	5	Reserved word
if	ifsym	6	Reserved word
fi	fisym	7	Reserved word
then	thensym	8	Reserved word
while	whilesym	9	Reserved word
do	dosym	10	Reserved word
od	odsym	11	Reserved word
call	callsym	12	Reserved word
const	constsym	13	Reserved word
var	varsym	14	Reserved word
procedure	procsym	15	Reserved word
write	writesym	16	Reserved word
read	readsym	17	Reserved word
else	elsesym	18	Reserved word

Special symbols See Table 5 for the complete list of special symbols. These begin at token number 19.

Table 5: Special Symbols Table

Symbol	Token Symbol	Token Number	Meaning/String Match
+	plussym	19	Addition operator
-	minussym	20	Subtraction operator
*	multsym	21	Multiplication operator
/	slashsym	22	Division operator
=	eqsym	23	Equality operator
<>	neqsym	24	Not equal operator ($\neq$)
<	lessym	25	Less than operator
<=	leqsym	26	Less than or equal ($\leq$)
>	gtrsym	27	Greater than operator
>=	geqsym	28	Greater than or equal ($\geq$)
(	lparentsym	29	Left parenthesis
)	rparentsym	30	Right parenthesis
,	commasym	31	Comma separator
;	semicolonsym	32	Semicolon
.	periodsym	33	Period/dot
:=	becomessym	34	Assignment operator

B.1.2 Ignored Elements

Comments Multi-line comments begin with `/*` and end with `*/`. During lexical analysis, the lexical analyzer recognizes the delimiters and ignores all content between them. See

Table 6 for details.

Table 6: Comment Format Definitions (Multi-line Comments)

Start Delimiter	End Delimiter	Description
/*	*/	Block comment

Invisible characters Tab, whitespace, and newline characters are skipped and not tokenized. See Table 7 for details.

Table 7: Invisible Characters (Ignored During Lexical Analysis)

Character	Description
Space	Space character (ignored)
Tab	Horizontal tab (ignored)
Newline	Line feed character (ignored)
Carriage Return	Carriage return character (ignored)

B.2 Name Table

The lexical analyzer maintains a name table for identifier management. The name table is not a token table; it is a compact mapping used to store identifier lexemes with a stable index.

Each name-table entry contains exactly two fields:

1. **Index:** a unique integer assigned when the identifier is first encountered.
2. **Identifier Name:** the exact identifier lexeme string.

Only identifier tokens (`identsym`) are stored in the name table. If an identifier is seen again, the existing index should be reused.

Table 8: Name Table Entry Format

Field	Description
Index	Identifier position
Name	Identifier string

In the console output, print the name table before the token list as an additional section. See the sample output in Appendix C for the Name Table layout.

B.3 TokenType Enumeration

The following C `typedef enum` defines all recognized symbols in the PL/0 lexical analyzer. Each token is assigned a unique numeric value corresponding to the token numbers specified in the recognition tables above. This enumeration can be used directly in your C implementation to represent token types.

TokenType Enumeration in C

```
typedef enum {
    skipsym = 1,     // Skip/ignore token
    identsym,        // Identifier
    numbersym,       // Number
    beginsym,        // begin
    endsym,          // end
    ifsym,           // if
    fisym,           // fi
    thensym,         // then
    whilesym,        // while
    dosym,           // do
    odsym,           // od
    callsym,         // call
    constsym,        // const
    varsym,          // var
    procsym,         // procedure
    writesym,        // write
    readsym,         // read
    elsesym,         // else
    plussym,         // +
    minussym,        // -
    multsym,         // *
    slashsym,        // /
    eqsym,           // =
    neqsym,          // <>
    lessym,          // <
    leqsym,          // <=
    gtrsym,          // >
    geqsym,          // >=
    lparentsym,      // (
    rparentsym,      // )
    commasym,        // ,
    semicolonsym,    // ;
    periodsym,       // .
    becomessym       // :=
} TokenType;
```

B.4 Lexical Error Reporting

The lexical analyzer must detect and report lexical errors, but it does not need to report grammar errors, syntax errors, or semantic errors. Your program should scan through the entire input program and report all lexical errors encountered. The following three types of lexical errors must be detected:

1. **Identifier too long:** Identifiers exceeding 11 characters
2. **Number too long:** Numbers exceeding 5 digits
3. **Invalid symbols:** Characters that are not part of the PL/0 language specification

C Complete Example for Correct Input

This appendix provides a complete, correct HW4 example showing the entire pipeline from PL/0 source code through the integrated scanner/parser/code generator. The example exercises all of the HW4-specific grammar features: a `procedure` declaration with its own `var` declaration and body, a `call` statement in the main block, and nested lexicographical levels (level 0 for the main program and level 1 inside procedure `x`).

C.1 Input: PL/0 Source Code

The following PL/0 program is the input file that `parsercodegen_comp` reads. It declares two variables at the main level, a single nested procedure `x` that declares two of its own local variables, and a main body that assigns a value and calls the procedure.

Input File

```
var a, b;
procedure x;
    var q, w;
    begin
        q := 1;
    end;
begin
    a := 2;
    call x;
end.
```

C.2 Terminal Output

After running `./parsercodegen_comp input.txt`, your program outputs the following to the terminal. This output shows both the generated PM/0 assembly code and the final state of the symbol table (after the main block has finished parsing, so every variable and nested procedure has been marked out of scope).

Terminal Output

```
Assembly Code:
+------+-----+---+-----+
| Line | OP  | L | M   |
+------+-----+---+-----+
| 0    | JMP | 0 | 18  |
| 1    | JMP | 0 | 6   |
| 2    | INC | 0 | 5   |
| 3    | LIT | 0 | 1   |
| 4    | STO | 0 | 3   |
| 5    | OPR | 0 | 0   |
| 6    | INC | 0 | 5   |
| 7    | LIT | 0 | 2   |
| 8    | STO | 0 | 3   |
| 9    | CAL | 0 | 3   |
| 10   | SYS | 0 | 3   |
+------+-----+---+-----+

Symbol Table:
+------+-------------+-------+-------+---------+------+
| Kind | Name        | Value | Level | Address | Mark |
+------+-------------+-------+-------+---------+------+
| 2    | a           | 0     | 0     | 3       | 1    |
| 2    | b           | 0     | 0     | 4       | 1    |
| 3    | x           | 0     | 0     | 3       | 1    |
| 2    | q           | 0     | 1     | 3       | 1    |
| 2    | w           | 0     | 1     | 4       | 1    |
+------+-------------+-------+-------+---------+------+
```

What to notice in the symbol table:

- `a` and `b` are variables (`kind = 2`) at level 0 with consecutive addresses starting at 3 (the first slot after the activation-record header).
- `x` is a procedure (`kind = 3`) at level 0. Its `addr` field holds the code address used as the M field of the matching `CAL` instruction; its `val` field is unused and shown as 0.
- `q` and `w` are variables (`kind = 2`) at level 1, because they are declared inside the body of procedure `x`. Their addresses restart at 3 because each procedure has its own activation record.
- Every entry has `mark = 1` because the whole program has finished parsing by the time the symbol table is printed, and the marking algorithm has marked every variable and nested procedure out of scope (constants, if any had been present, would *not* have been marked).

C.3 File Output: elf.txt

In addition to terminal output, `parsercodegen_comp` must write the executable code to `elf.txt`. This file contains the same instructions but in numeric format (opcode, L, M) without line numbers or mnemonics:

File Output: elf.txt

```
7 0 18
7 0 6
6 0 5
1 0 1
4 0 3
2 0 0
6 0 5
1 0 2
4 0 3
5 0 3
9 0 3
```

Important Notes:

- Each line contains three space-separated integers: opcode, L-value, and M-value.
- This file format is required for the PM/0 virtual machine to execute your code.
- Refer to Appendix A for the complete ISA specification and opcode mappings.

C.4 How to Test Your Output

Hint: The `elf.txt` produced by your `parsercodegen_comp` should be **directly runnable by the PM/0 virtual machine you wrote in HW1**. If your HW1 VM accepts `elf.txt` as input and executes the program to completion (for this example, halting cleanly after the call to `x` returns and the main block reaches its `SYS 0 3` HALT), that is a strong sign the code you are generating is consistent with the ISA. If your HW1 VM instead rejects the file, crashes, runs off the end of the code segment, or produces wrong results, then something in the pipeline is wrong.

Important caveat: This testing strategy *assumes your HW1 VM is itself 100% correct.* If your HW1 VM has bugs, a correct HW4 `elf.txt` may still misbehave when executed (false negative), and a buggy HW4 `elf.txt` may still appear to run (false positive). Before trusting the VM round-trip as a signal:

- Re-run your HW1 VM against your own HW1 test cases and confirm they still pass; if any HW1 test now fails, fix the VM before using it to judge HW4.
- Where possible, also execute the `elf.txt` with an instructor-provided or known-good PM/0 VM and compare the behavior.

- If the two VMs disagree, trust the known-good VM and treat your HW1 VM as suspect.
- Do not "fix" HW4 code to make a buggy HW1 VM happy. Doing so will make HW4 fail when the grader runs it through a correct VM.

Assuming your HW1 VM is correct, typical HW4-side things to check first when the round-trip fails are:

- Is the opening `JMP` correctly targeting the first instruction of the main block (and *not* the procedure's body)?
- Does every procedure body end with `OPR 0 0` (`RTN`)?
- Does every `CAL` use the `addr` field stored in the target procedure's symbol-table entry?
- Are the L fields of your `LOD`, `STO`, and `CAL` instructions equal to the difference currentLevel − `symbol.level` at the reference site?

Used with this caveat in mind, running the generated `elf.txt` through a trusted PM/0 VM is one of the fastest ways to check HW4 correctness before submission.

D Scanning Error Example

This appendix demonstrates how your implementation should handle input files containing **lexical/scanning errors**. The key principle is that the scanner (integrated within `parsercodegen_comp.c`) continues processing the entire input even when errors are detected, but represents errors as a special error token (token type 1). The parser then detects this error token and halts immediately.

D.1 Input: PL/0 Source Code with Error

The following PL/0 program contains a lexical error (identifier name too long):

Input File with Error

```
const z = 5;
var xcjioasunihujacioj, y;
begin
        xcjioasunihujacioj := y * 2;
end.
```

Error: The identifier `xcjioasunihujacioj` is 18 characters long, which exceeds the maximum allowed length of 11 characters.

D.2 Terminal Output

After running `./parsercodegen_comp input.txt`, the terminal displays:

Terminal Output for Error

```
Error: Scanning error detected by lexer (skipsym present)
```

D.3 File Output: elf.txt

The `elf.txt` file contains the same error message:

elf.txt for Error

```
Error: Scanning error detected by lexer (skipsym present)
```

D.4 Key Principle

All three scanner errors produce the same unified error message from the parser. The parser does not need to distinguish between different types of lexical errors—it only needs to detect that an error occurred (presence of token type 1) and halt with the appropriate message.

E Syntax Error Example

The syntax-error walkthrough below exercises only an HW3-era grammar violation (missing period). The procedure/call-related error walkthroughs will be added once the reference implementation of `parsercodegen_comp.c` is finalized.

This appendix demonstrates how your implementation should handle **syntax errors** (grammar violations). Unlike Appendix D which dealt with **scanning errors** (lexical errors), this appendix focuses on **syntax errors** (grammar violations detected during parsing).

E.1 Key Distinction: Scanning vs. Syntax Errors

- **Scanning Errors (Appendix D):** Lexical errors detected by the scanner (e.g., identifier too long, number too long, invalid symbols). The scanner represents these as token type 1 (skipsym), and the parser detects the presence of this error token.
- **Syntax Errors (This Appendix):** Grammar violations detected by the parser during parsing (e.g., missing period, missing semicolon, incorrect statement structure). The scanner produces valid tokens, but the token sequence does not match the grammar rules.

E.2 Input: PL/0 Source Code with Syntax Error

The following PL/0 program contains a syntax error (missing period at the end):

Input File with Missing Period

```
var x, y;
begin
    read y;
    x := y * 2;
end
```

Error: According to the grammar specification (see Figure 1), a program must follow the production:

```
<program> ::= <block> "."
```

The period symbol after `end` is **required** but missing in this input.

E.3 Terminal Output

After running `./parsercodegen_comp input.txt`, the terminal displays:

Terminal Output for Syntax Error

```
Error: program must end with period
```

E.4 File Output: elf.txt

The `elf.txt` file contains the same error message:

elf.txt for Syntax Error

```
Error: program must end with period
```

E.5 Key Principle

The scanner handles lexical errors by emitting error tokens (token type 1). The parser handles both scanning errors (by detecting token type 1) and syntax errors (by detecting grammar violations during parsing). Each type of error produces a specific, appropriate error message.

F Symbol Table Structure

This appendix provides the recommended data structure for implementing the symbol table in your `parsercodegen_comp.c` implementation.

F.1 Recommended Symbol Table Structure

Symbol Table Data Structure - RECOMMENDED

Important: While you may use alternative data structures (e.g., linked lists, dynamically allocated arrays), the **structure fields must remain the same** as shown below. The autograder expects these specific field names and types.

```
#define MAX_SYMBOL_TABLE_SIZE 500

typedef struct {
    int kind;       // const = 1, var = 2, proc = 3
    char name[12];  // name up to 11 chars
    int val;        // number (ASCII value)
    int level;      // L level
    int addr;       // M address
    int mark;       // to indicate unavailable or deleted
} symbol;

symbol symbol_table[MAX_SYMBOL_TABLE_SIZE];
```

Field Descriptions:

- `kind`: Type of symbol (1 = constant, 2 = variable, 3 = procedure).
- `name[12]`: Symbol name (up to 11 characters plus null terminator).
- `val`: For *constants*, the constant's integer value. For variables and procedures the semantics of `val` are described in the per-kind subsections below.
- `level`: Lexicographical level (the L value that appears in PM/0 `LOD`, `STO`, and `CAL` instructions *references* this symbol, via a level difference). HW4 requires this field to reflect the actual nesting depth of the declaration (see below).
- `addr`: For variables, the M offset within the activation record. For procedures, the starting code address of the procedure body (used as the M field of the `CAL` that targets it). Unused for constants.
- `mark`: Flag indicating whether the symbol is currently in scope (0) or has been marked out of scope (1) because its enclosing block has finished parsing.

F.2 Symbol Table Size

- **Maximum size:** 500 symbols (`MAX_SYMBOL_TABLE_SIZE = 500`)
- **Sufficiency:** A table size of 500 is sufficient for all test cases in this assignment
- **Recommendation:** Use the simple array-based structure shown above for ease of implementation
- **Alternative structures:** You may use linked lists or dynamically allocated memory if you prefer, but the structure fields must remain the same

F.3 Lexicographical Level Tracking (HW4)

Unlike HW3, not every symbol in an HW4 program lives at level 0. *Variables and procedures* record the lexicographical level of the block in which they are declared: level 0 for the main program, level 1 for a procedure declared inside the main program, level 2 for a procedure declared inside that procedure, and so on. Your parser must carry a *current level* counter that starts at 0 for the program's outermost block, is incremented by one each time a nested procedure's block is entered, and is decremented when that block is exited.

Constants are different. The lexicographical level of a constant symbol is *always* 0, regardless of where the constant is declared. A constant is effectively a compile-time literal: once inserted into the symbol table it remains globally accessible for the rest of the program, and the code generator emits it as a `LIT` instruction rather than a `LOD`, so no L field is involved.

When a reference site needs to access a *variable*, the L field of the emitted `LOD` or `STO` instruction is the difference:

$$L_{\text{emit}} = \text{currentLevel} - \texttt{symbol.level}$$

and similarly for the L field of a `CAL` instruction that targets a procedure symbol. This difference is always ≥ 0 because a block can only reference variables and procedures from itself or an enclosing scope.

F.4 Marking Algorithm (HW4)

When a block finishes parsing, every *variable* and every *nested procedure* that was declared inside that block must have its `mark` field set to 1. A symbol with `mark = 1` is considered *out of scope* and must not match any subsequent lookup. This is how HW4 enforces the PL/0 scoping rule that variables and procedures declared inside a procedure are not visible outside of it.

Constants are excluded from marking. Because constants are always at level 0 and always accessible, they are *not* marked when their enclosing block finishes. A constant stays available for the remainder of the program.

Note: The marking algorithm is strictly additive. Once a symbol has been marked, it stays in the table for reference/debugging purposes but is invisible to the parser's lookup routine.

F.5 Required Storage for Different Symbol Types

F.5.1 For Constants (kind = 1)

When storing a constant symbol, you **must** populate the following fields:

Required Fields for Constants

- `kind = 1`
- `name`: The constant's identifier name
- `val`: The constant's integer value
- `level = 0` (constants are always at level 0 and remain globally accessible for the entire program)
- `mark = 0` (constants are *not* marked out of scope by the marking algorithm; see Section F.4)

Note: The `addr` field is not used for constants.

F.5.2 For Variables (kind = 2)

When storing a variable symbol, you **must** populate the following fields:

Required Fields for Variables

- `kind = 2`
- `name`: The variable's identifier name
- `level`: The lexicographical level where the variable is declared (0 for main program, increases with procedure nesting)
- `addr`: The variable's memory address (M value for `LOD`/`STO` instructions)
- `mark = 0` (initially available)

Note: The `val` field is not used for variables.

F.5.3 For Procedures (kind = 3)

When storing a procedure symbol, you **must** populate the following fields:

Required Fields for Procedures

- `kind = 3`
- `name`: The procedure's identifier name
- `level`: The lexicographical level where the procedure is declared (0 for main program, increases with procedure nesting)
- `addr`: The code address where the procedure begins (used as M value for `CAL` instruction)
- `mark = 0` (initially available)

Note: The `val` field is not used for procedures.

F.6 Symbol Table Operations

Your implementation should support the following operations:

- **Insert:** Add a new symbol to the table. When checking for duplicate names you must consider only symbols that are currently in scope (`mark == 0`).
- **Lookup:** Search for a symbol by name. Skip symbols with `mark == 1`; among the remaining matches, constants (which are always at level 0) are always visible, and for variables and procedures prefer the innermost match whose level is $\leq$ the current level.
- **Mark:** When a block finishes, set `mark = 1` on every variable and nested-procedure symbol that was declared inside that block. Constants are *not* marked (see Section F.4).

G Pseudocode

This appendix provides pseudocode for implementing the recursive-descent parser and code generator. The pseudocode closely follows the grammar structure and demonstrates the general approach for parsing and code generation.

Important: The pseudocode in this appendix is a starting point and is not 100% accurate. You must understand the grammar in Figure 1 and adapt the pseudocode before coding. Blindly translating this pseudocode into C will result in deductions and may be treated as plagiarism.

Each function below is presented in a separate box for clarity. Study each function carefully and compare it with the corresponding grammar production.

G.1 Symbol Table Helper Function

```
SYMBOLTABLECHECK (string)
    linear search through symbol table looking at name
    return index if found, -1 if not
```

G.2 PROGRAM Function

```
PROGRAM
    BLOCK
    if token != periodsym
        error
    emit HALT
```

G.3 BLOCK Function

```
BLOCK
    CONST-DECLARATION
    numVars = VAR-DECLARATION
    PROCEDURE-DECLARATION
    emit INC (M = 3 + numVars)
    STATEMENT
```

G.4 CONST-DECLARATION Function

```
CONST-DECLARATION
    if token == const
        do
            get next token
            if token != identsym
                error
            if SYMBOLTABLECHECK (token) != -1
                error
            save ident name
            get next token
            if token != eqlsym
                error
            get next token
            if token != numbersym
                error
            add to symbol table (kind 1, saved name, number, 0, 0)
            get next token
        while token == commasym
        if token != semicolonsym
            error
        get next token
```

G.5 VAR-DECLARATION Function

```
VAR-DECLARATION - returns number of variables
    numVars = 0
    if token == varsym
        do
            get next token
            if token != identsym
                error
            if SYMBOLTABLECHECK (token) != -1
                error
            add to symbol table (kind 2, ident, 0, 0, numVars + 3)
            numVars++
            get next token
        while token == commasym
        if token != semicolonsym
            error
        get next token
    return numVars
```

G.6 PROCEDURE-DECLARATION Function

```
PROCEDURE-DECLARATION
    while token == procsym
        get next token
        if token != identsym
            error
        get next token
        if token != semicolonsym
            error
        get next token
        add to symbol table (kind 3, ident, level, codeIndex)
        BLOCK(level + 1)
        if token != semicolonsym
            error
        get next token
```

G.7 STATEMENT Function

The STATEMENT routine handles seven statement forms. For readability we present it in three non-breakable boxes, one per group; in your implementation the boxes below correspond to a single `STATEMENT` function whose body is a sequence of `if` branches.

STATEMENT — assignment and call

```
STATEMENT
    if token == identsym
        symIdx = SYMBOLTABLECHECK (token)
        if symIdx == -1
            error
        if table[symIdx].kind != 2 (not a var)
            error
        get next token
        if token != becomessym
            error
        get next token
        EXPRESSION
        emit STO (M = table[symIdx].addr)
        return
    if token == callsym
        get next token
        if token != identsym
            error
        idx = SYMBOLTABLECHECK (token)
        if idx == -1
            error
        if symbolTable[idx].kind != 3
            error
        emit CAL (L, symbolTable[idx].addr)
        get next token
        return
```

STATEMENT — compound, if, and while

```
    if token == beginsym
        do
            get next token
            STATEMENT
        while token == semicolonsym
        if token != endsym
            error
        get next token
        return
    if token == ifsym
        get next token
        CONDITION
        jpcIdx = current code index
        emit JPC
        if token != thensym
            error
        get next token
        STATEMENT
        code[jpcIdx].M = current code index
        return
    if token == whilesym
        get next token
        loopIdx = current code index
        CONDITION
        if token != dosym
            error
        get next token
        jpcIdx = current code index
        emit JPC
        STATEMENT
        emit JMP (M = loopIdx)
        code[jpcIdx].M = current code index
        return
```

STATEMENT — read and write

```
    if token == readsym
        get next token
        if token != identsym
            error
        symIdx = SYMBOLTABLECHECK (token)
        if symIdx == -1
            error
        if table[symIdx].kind != 2 (not a var)
            error
        get next token
        emit READ
        emit STO (M = table[symIdx].addr)
        return
    if token == writesym
        get next token
        EXPRESSION
        emit WRITE
        return
```

G.8 CONDITION Function

```
CONDITION
    EXPRESSION
    if token == eqlsym
        get next token
        EXPRESSION
        emit EQL
    else if token == neqsym
        get next token
        EXPRESSION
        emit NEQ
    else if token == lessym
        get next token
        EXPRESSION
        emit LSS
    else if token == leqsym
        get next token
        EXPRESSION
        emit LEQ
    else if token == gtrsym
        get next token
        EXPRESSION
        emit GTR
    else if token == geqsym
        get next token
        EXPRESSION
        emit GEQ
    else
        error
```

G.9 EXPRESSION Function

```
EXPRESSION
    TERM
    while token == plussym || token == minussym
        if token == plussym
            get next token
            TERM
            emit ADD
        else
            get next token
            TERM
            emit SUB
```

G.10 TERM Function

```
TERM
    FACTOR
    while token == multsym || token == slashsym
        if token == multsym
            get next token
            FACTOR
            emit MUL
        else
            get next token
            FACTOR
            emit DIV
```

G.11 FACTOR Function

```
FACTOR
    if token == identsym
        symIdx = SYMBOLTABLECHECK (token)
        if symIdx == -1
            error
        if table[symIdx].kind == 1 (const)
            emit LIT (M = table[symIdx].value)
        else (var)
            emit LOD (M = table[symIdx].addr)
        get next token
    else if token == numbersym
        emit LIT
        get next token
    else if token == lparentsym
        get next token
        EXPRESSION
        if token != rparentsym
            error
        get next token
    else
        error
```